

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবিয়ার

(Prof. Chester Rebeiro)

কম্পিউটার সায়েন্স এন্ড ইঞ্জিনিয়ারিং

(Department of Computer Science and Engineering)

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি (Indian Institute of Technology), মাদ্রাজ

সপ্তাহ – 01 বক্তৃতা- 19

অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম

এখন পর্যন্ত আমরা কিছু সিডিউলিং অ্যালগরিদম দেখেছি যেমন, ফার্স্ট কাম ফার্স্ট সার্ভ এবং প্রিএম্পটিভ সিডিউলিং অ্যালগরিদম যেমন রাউন্ড রবিন। তাই এই বিশেষ ভিডিওতে, আমরা অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম হিসাবে পরিচিত যা সিডিউলিং এর জন্য অ্যালগরিদম টা দেখবো। সুতরাং আমরা একটি উদাহরণের সঙ্গে এই বক্তৃতা শুরু হবে।

(স্লাইডটাইপ পড়ুন: 00:40)

সুতরাং, আসুন আমরা এই বিশেষ উদাহরণের উপর নির্ভর করি যা আমরা শেষ ভিডিওতে নিয়েছি। তাই আমরা এই রাউন্ড রবিন সিডিউলিং অ্যালগরিদম দেখেছি যেখানে আমরা P4 থেকে P1 চারটি প্রসেস নিয়েছি এবং এই প্রসেসগুলি 0, 2, 3 এবং 9 এর বিভিন্ন সময়ে এসেছে, এবং তাদের বিভিন্ন বার্ট টাইম যথাক্রমে 7, 4, 2, এবং 1 চক্রের আছে। এখন আমরা জড়িত প্রসেসের উপরে একটি বিট যোগ করব। আসুন আমরা অনুমান করি P2 একটি জটিল প্রসেস, যখন P1, P3 এবং P4 কম ক্রিটিক্যাল। সুতরাং, একটি ক্রিটিক্যাল প্রসেস কি? সুতরাং আসুন একটি অপারেটিং সিস্টেমের একটি উদাহরণ দিন যা একটি গাড়িতে উপস্থিত একটি মাইক্রো প্রসেসর চলতে থাকে, এবং যখনই বিরতিগুলি চাপা হয় তখন প্রসেসটি P2 এক্সিকিউট হবে এবং এটি গাড়ির সাথে সম্পর্কিত কিছু কাজ করবে, যেখানে প্রসেস P1, P3 এবং P4 কম ক্রিটিক্যাল কাজ। উদাহরণস্বরূপ প্রসেস P1, P3 এবং P4- আবারও মিউজিক প্লেয়ারের প্রসেস হতে পারে, সুতরাং একটি প্রসেস মিউজিক প্লেয়ার হতে পারে, অন্যটি এসি বা হিটার নিয়ন্ত্রণ করতে পারে ইত্যাদি। সুতরাং, আবার এই ন্যাট চার্ট এর দিকে তাকান, আমরা দেখছি যে P2 দ্বিতীয় চক্রে

আসে এবং এটি এক্সিকিউশনের সম্পূর্ণ চক্র যথেষ্ট পরিমাণ লাগে। এর মূল কারণ হচ্ছে এটি একটি রাউন্ড রবিন সিডিউলিং স্কিম এবং অন্যান্য প্রসেসের CPU এর একটি ন্যায় ভাগ রয়েছে। সুতরাং, P2 প্রথমে এখানে এক্সিকিউট হয় এবং তারপর P1 এবং P3 এক্সিকিউট হয়, তারপর P2 আবার বার বার এক্সিকিউট হয় এবং অবশেষে এই নির্দিষ্ট সময়ে তার বার্ষ্ট এক্সিকিউশন সমাপ্ত করে। এখন, আপনি দেখবেন এই নির্দিষ্ট স্কিমের সঙ্গে একটি সুস্পষ্ট সমস্যা আছে। যেহেতু, P2 একটি জটিল প্রসেস যা গাড়ীটি ভেঙ্গে নিয়ন্ত্রণ করার জন্য এটি দীর্ঘ সময় ধরে এটি কার্যকর করার জন্য এটি কার্যকর করেছে, এবং এর ফলে বিপর্যয়কর দুর্ঘটনা হতে পারে।

(স্লাইডসময় পড়ুন: 02:49)

সুতরাং এই মত কিছু আশা করা হবে। যখনই P 2 এর মতো একটি জটিল প্রসেস রেডি স্টেটে আসে, তখন এটি একটি অগ্রাধিকার পাবে এবং এটি ক্রমাগতভাবে এক্সিকিউট করতে সক্ষম হওয়া উচিত, তবুও এটি সত্য যে, কিউতে বিদ্যমান নিম্ন অগ্রাধিকারের সাথে অন্যান্য প্রসেসগুলিও আছে। তাই আমাদের আরো বাস্তববাদী পরিস্থিতি চাই। সুতরাং সিডিউলিং আলগোরিদমগুলি যা প্রসেসগুলির মধ্যে অগ্রাধিকারসমূহের যত্ন করে অগ্রাধিকার ভিত্তিক নির্ধারিত অ্যালগোরিদম হিসাবে পরিচিত, এবং আমরা এই বিশেষ ভিডিও বক্তৃতাতে সেটা দেখবো।

(স্লাইডসময় পড়ুন: 03:28)

একটি অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদমে প্রতিটি প্রসেসের একটি অগ্রাধিকার সংখ্যা বরাদ্দ করা হয়। একটি অগ্রাধিকার সংখ্যা একটি নির্দিষ্ট পরিসীমার মধ্যে একটি সংখ্যা মাত্র, তাই এই পরিসীমা অপারেটিং সিস্টেম দ্বারা পূর্বনির্ধারণ হয়। উদাহরণস্বরূপ, অগ্রাধিকারের পরিসর 0 এবং 255 এর মধ্যে হতে পারে। তাই, কার্যকর করা প্রতিটি প্রসেসের 0 এবং 255 এর মধ্যে একটি অগ্রাধিকার সংখ্যা নির্ধারণ করা হবে। একটি সংখ্যা যা ছোট, এটি একটি অগ্রাধিকার সংখ্যা যা ছোট যা 0 এর কাছাকাছি যার অর্থ এই প্রসেসটি একটি উচ্চ অগ্রাধিকার প্রসেস বা একটি উচ্চ অগ্রাধিকার টাস্ক। অন্য দিকে, একটি বড় অগ্রাধিকার সংখ্যা মানে এই প্রসেসের কম অগ্রাধিকার রয়েছে। তাই এটি শুধু একটি নামকরণ যা লিনাক্স অনুসরণ করে, অন্য অপারেটিং সিস্টেমের একটি ভিন্ন পদ্ধতি থাকতে পারে। তাই এটি উদাহরণস্বরূপ একটি উচ্চ অগ্রাধিকার প্রসেস নির্দেশ করে একটি উচ্চ নম্বর রাখা হবে, যখন একটি নিম্ন অগ্রাধিকার সংখ্যা কম অগ্রাধিকার প্রসেস ইঙ্গিত করে, কিন্তু এই বক্তৃতার জন্য আমরা নেব একটি ছোট অগ্রাধিকার সংখ্যা একটি উচ্চ অগ্রাধিকার প্রসেস মানে এবং বিপরীতভাবে লিনাক্স নামকরণের সঙ্গে যায়। এখন, একটি অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম, যদিও রেডি কিউতে বেশ কিছু কাজ উপস্থিত থাকতে পারে, সিডিউলিং আলগোরিদম সর্বোচ্চ অগ্রাধিকারের সাথে কাজটি বেছে নেবে। অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদমগুলির সুবিধাটি খুবই সহজ। মূলত একটি অপারেটিং সিস্টেম যা একটি অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম সমর্থন করে, সম্পর্কযুক্ত কাজগুলিকে গুরুত্ব দেওয়া যেতে পারে অথবা

প্রসেসগুলিকে প্রাসঙ্গিক গুরুত্ব দেওয়া যেতে পারে। উচ্চ অগ্রাধিকার প্রসেসগুলি আসলে CPU- এ কম অগ্রাধিকার প্রসেসর তুলনায় উচ্চতর অগ্রাধিকার পাবে। অন্য দিকে, একটি অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম থাকার প্রধান ত্রুটি হল যে এটি স্টার্ভেশন নামে পরিচিত হতে পারে। তাই মূলত, একটি স্টার্ডড প্রসেস এমন একটি প্রসেস যা CPU এ কখনো এক্সিকিউট করতে সক্ষম হয় না, যদিও এটি একটি দীর্ঘ সময়ের জন্য প্রস্তুত সারিতে আছে। এটা দেখতে খুব সহজ যে স্টার্ভেশন প্রধানত কম অগ্রাধিকার প্রসেসের জন্য ঘটে।

(স্লাইডটাইপ পড়ুন: 05:39)

সুতরাং আসুন আমরা একটি উদাহরণ দিয়ে দেখি কিভাবে স্টার্ভেশন দেখা দেয়। তাই আমরা একই চার প্রসেস P1 থেকে P4 নেব যেগুলি 0, 2, 3 এবং 9 এর বিভিন্ন সময়ে আসবে এবং বিভিন্ন বার্ট সময়ে 8, 4, 2 এবং 1 থাকবে। এছাড়াও, অনুমান করি যে P1, P2 এবং P3 এর অগ্রাধিকার রয়েছে P4 থেকে বা অন্য শব্দে P4 সর্বনিম্ন অগ্রাধিকার প্রসেস। আমরা এই অনুমান করি যে প্রতি 15 চক্রে এই বিশেষ ক্রম পুনরাবৃত্তি হচ্ছে, যে প্রসেসরের 15 চক্র পরে, P1 আবার আসে, P2 আবার আসে, P3 এবং একইভাবে P4 আসে, তাই প্রসেসের এই ক্রম ধারাবাহিকভাবে অনবরত রেডি স্টেটের মধ্যে আসছে। তাই আমরা দেখি যে, যদি আপনি ন্যাট চার্ট দেখেন যে অগ্রাধিকার এর উপর নির্ভর করে P1, P2 এবং P3 সিডিউলিং হয়। যাইহোক, কম অগ্রাধিকার টাস্ক 15 চক্রের মধ্যে কখনোই সিডিউলিং হয় না। উপরন্তু, P1, P2, P3 এবং P4 15 টি চক্র পরে পুনরায় কিউয়ের মধ্যে আবার ফিরে আসে। সুতরাং, আবার P1, P2 এবং P3 সিডিউল সময় নির্ধারণ করা হলে P4 কখনই নির্ধারিত হবে না। তাই এর ফলাফল হিসাবে আমরা দেখি যে, এই সর্বনিম্ন অগ্রাধিকার প্রসেস স্টার্ভেশন হয়ে যায়। মূলত, যদিও এটি রেডি কিউতে আছে এবং কেবলমাত্র CPU কে বরাদ্দ করার জন্য অপেক্ষা করে, এটি কখনোই CPU- এ স্থানান্তরিত সুযোগ পায় না কারণ সর্বদা উচ্চতর অগ্রাধিকারের প্রসেসগুলি রেডি কিউতে পৌঁছয়।

(স্লাইডটাইপ পড়ুন: 07:28)

তাই আসুন দেখি কিভাবে অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম স্টার্ভেশনকে সামাল দেয়। তাই মূলত, অপারেটিং সিস্টেমগুলির মধ্যে একটি অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম রয়েছে, যাতে নির্ধারিত সময়সূচী সীমাবদ্ধভাবে প্রসেসের অগ্রাধিকারকে সামঞ্জস্য করে তা নিশ্চিত করতে হবে যে সমস্ত প্রসেসগুলি CPU- এ শেষ হবে। এর মানে এই যে, আমাদের উদাহরণে প্রসেস P4 এর অগ্রাধিকার ধীরে ধীরে একটি নির্দিষ্ট সময়ের মধ্যে বাড়িয়ে তুলবে যতক্ষণ পর্যন্ত এটি প্রসেস P1, P2 এবং P3 এর অগ্রাধিকারের চেয়ে বড় বা সমান হবে না। সময় প্রসেস P4 CPU- এ চালানো একটি সুযোগ পাবেন। এইভাবে, আমরা সময়ের সাথে সাথে P4 প্রসেসকরণ দ্বারা দেখি যে স্টার্ভেশনে যে অন্যান্য ঘটেছিল, যেহেতু পি 4 একটি কম অগ্রাধিকার প্রসেস টা দূর হল। তাই অনেক কৌশল আছে যা অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম স্টার্ভেশন মোকাবেলা করতে পারে। সুতরাং আমরা এখানে একটি বিশেষ উদাহরণ দিতে পারি যা একটি অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম

সহ একটি অপারেটিং সিস্টেম নিশ্চিত করতে পারে যে প্রতিটি প্রসেস, এমনকি কম অগ্রাধিকার প্রসেসও শেষ পর্যন্ত CPU এ এক্সিকিউট হবে। সুতরাং, আমাদের এই বিশেষ অপারেটিং সিস্টেমের কথা বলুন যখন একটি প্রসেস শুরু হয় একটি বেস অগ্রাধিকার দেওয়া এখন, এই বেস অগ্রাধিকার প্রসেস থেকে প্রসেস পৃথক হবে, একটি উচ্চ অগ্রাধিকার প্রসেসকে একটি উচ্চ অগ্রাধিকার সংখ্যা দেওয়া হবে, একটি কম অগ্রাধিকার প্রসেসকে একটি কম অগ্রাধিকার সংখ্যা দেওয়া হবে। এখন, প্রতিটি সময় স্লট বা টাইম স্লাইড পরে প্রসেস অগ্রাধিকার একটি নির্দিষ্ট মান দ্বারা বৃদ্ধি করা হয়। তাই, প্রাথমিকভাবে রেডি কিউয়ের সমস্ত প্রসেসগুলি তাদের অগ্রাধিকার একটি নির্ধারিত মান দ্বারা বাড়িয়েছে যে প্রসেসটি বর্তমানে সিপিইউতে চালানো হচ্ছে। তাই আপনি দেখতে পাবেন যে, সময়কালের মধ্যে অগ্রাধিকার লাইনের সমস্ত প্রসেসগুলি ধীরে ধীরে তাদের অগ্রাধিকার বৃদ্ধি পাবে। এখন এমনকি একটি কম অগ্রাধিকার প্রসেস তার অগ্রাধিকার পয়েন্ট পর্যন্ত বৃদ্ধি হবে যে তার অগ্রাধিকার যথেষ্ট উচ্চ যাতে এটি CPU তে সিডিউল হয়। সুতরাং, এটি CPU- র মধ্যে সিডিউল হওয়ার পরে এবং CPU- র মধ্যে সঞ্চালিত হওয়ার পরে, তার সময় স্লাইস শেষে তার অগ্রাধিকার তার বেস অগ্রাধিকার ফিরে রিসেট করা হয়। এইভাবে যা 250 এর বেস অগ্রাধিকার সঙ্গে শুরু হয় ধীরে ধীরে তার অগ্রাধিকার বৃদ্ধি করা হবে, তাই এটি একটি বিন্দু যা চালানো শুরু হয় এবং এটি চালানো শুরু করার পর তার অগ্রাধিকার 250 পুনরায় সেট করা হয়। সুতরাং, স্টার্ভেশন এড়ানো যাবে।

(স্লাইডসময় পড়ুন: 10:20)

তাই এই উপর ভিত্তি করে, একটি প্রসেসের জন্য উপস্থিত দুই প্রকারের অগ্রাধিকার আছে। স্ট্যাটিক অগ্রাধিকার যা সাধারণত এক্সিকিউশন শুরুতে সেট করা হয়, তাই এটি ব্যবহারকারী দ্বারা সেট করা যায় যে অ্যাপ্লিকেশন শুরু করা হয়, এবং সুযোগ দ্বারা যদি ব্যবহারকারী যে নির্দিষ্ট অগ্রাধিকার সেট না করে তাহলে একটি ডিফল্ট মান নেওয়া হয়। অন্যদিকে, দ্বিতীয় প্রকারের অগ্রাধিকারটি ডায়নামিক অগ্রাধিকার হিসাবে পরিচিত, যেখানে সিডিউলার প্রসেসের অগ্রাধিকার পরিবর্তন করতে পারেনকিছু নির্দিষ্ট সময়সূচী অর্জন করার জন্য, এক্সিকিউশন চালাকালীন। উদাহরণস্বরূপ, সিডিউলার একটি প্রসেস অগ্রাধিকার হ্রাস করার জন্য আরেকটি প্রসেস চালানোর একটি সুযোগ দিতে পারে। আরেকটি উদাহরণ যা আপনি আগে দেখেছেন I / O বাউন্ড প্রসেসগুলির অগ্রাধিকার বৃদ্ধি করা। যেহেতু, I / O বাউন্ড প্রসেসগুলি সাধারণত একটি দ্রুত রেসপন্স টাইম প্রয়োজন।

(স্লাইডটাইপ পড়ুন: 11:19)

একটি পরিসীমা থাকার একটি অসুবিধা হল যে একটি প্রসেস তার অগ্রাধিকার প্রাপ্ত হয়, সেক্ষেত্রে যখন সিস্টেমে উপস্থিত প্রসেসের একটি বড় সংখ্যা আছে। এই ক্ষেত্রে, একই প্রসেসগুলি একই অগ্রাধিকারের সাথে সঞ্চালন করা হতে পারে এমন অনেকগুলি প্রসেস হতে পারে। উদাহরণস্বরূপ, আসুন আমরা ধরি যে একটি অগ্রাধিকারের

পরিসর 0 থেকে 255 এর মধ্যে এবং সিস্টেমে দশ হাজারের বেশি প্রসেস চলছে। এই ক্ষেত্রে প্রতিটি অগ্রাধিকার সংখ্যাটি সেই অগ্রাধিকার সংখ্যাটি দ্বারা সম্পাদিত বিভিন্ন প্রসেস হতে পারে। তাই এই এই বিশেষ চিত্রে চিত্রিত করা হয়, যেখানে আমরা চারটি প্রসেস বিভিন্ন সময়ে আসছে এবং বিভিন্ন বাস্ট সময়য়ে আছে, কিন্তু তাদের সব একই অগ্রাধিকার আছে। সুতরাং এই ক্ষেত্রে, সিডিউলিং নন প্রায়োরীটি সিডিউলিং অ্যালগরিদমের মত আচরণ করা শুরু হবে। উদাহরণস্বরূপ এটি একটি রাউন্ড রবিনের সিডিউলিং অ্যালগরিদমের মত আচরণ করতে শুরু করবে।

(স্লাইডটাইপ পড়ুন: 12:২২)

তাই এটা হ্যান্ডেল করার জন্য, আধুনিক অপারেটিং সিস্টেমগুলির মধ্যে মাল্টিলেভেল কিউ আছে যা প্রত্যেক প্রসেসের একমাত্র অগ্রাধিকার সংখ্যা প্রদানের পরিবর্তে; প্রসেসগুলি একটি অগ্রাধিকার ক্লাসে নিযুক্ত করা হয়। এখন অপারেটিং সিস্টেমের মধ্যে বেশ কয়েকটি অগ্রাধিকার ক্লাস থাকতে পারে এবং এই অগ্রাধিকার ক্লাসগুলির প্রত্যেকটি উচ্চতর অগ্রাধিকার থেকে কম অগ্রাধিকার হতে পারে। উদাহরণস্বরূপ, একটি বাস্তব সময় অগ্রাধিকার ক্লাস, একটি সিস্টেম অগ্রাধিকার ক্লাস এবং ইন্টারেক্টিভ অগ্রাধিকার ক্লাস বা ব্যাচ অগ্রাধিকার ক্লাস এবং তাই হতে পারে। সমস্ত প্রাইমারি প্রসেস এই অগ্রাধিকার শ্রেণীতে উপস্থিত হবে। সুতরাং এই অগ্রাধিকার ক্লাসের প্রতিটিতে একটি রেডি কিউ আছে এবং যখন একটি বাস্তব সময় প্রসেস এক্সিকিউট করতে প্রস্তুত এটি এই বিশেষ রেডি কিউতে যোগ করা হয়, এটি বাস্তব সময় ক্লাস অগ্রাধিকারে যোগ করা হয়। অনুরূপভাবে, যদি কোনও ইন্টারেক্টিভ প্রসেস এক্সিকিউট হতে চায় এবং এটি এক্সিকিউটের জন্য প্রস্তুত থাকে তবে ইন্টারেক্টিভ অগ্রাধিকার ক্লাসে এই রেডি কিউতে যুক্ত হয়ে যায়। এখন, টাইম স্লাইসের শেষে সিডিউলিং যখন এক্সিকিউট হয়, সিডিউলার সর্বাধিক অগ্রাধিকার শ্রেণীতে থাকে যা কমপক্ষে একটি রেডি প্রসেস থাকে। সুতরাং উদাহরণস্বরূপ, সিডিউলার সর্বোচ্চ অগ্রাধিকার থেকে সর্বনিম্ন অগ্রাধিকার পর্যন্ত এই অগ্রাধিকার ক্লাস স্ক্যান করা হবে, এবং এটি যে নির্দিষ্ট অগ্রাধিকার বর্গ নির্বাচন করবে যা প্রস্তুত করে অন্তত একটি প্রসেস রেডি কিউতে আছে। এখন, যদি রেডি কিউতে ঠিক একটি প্রসেস থাকে তবে আমাদের কোন সমস্যা নেই, এখন ওই প্রসেসটি CPU এ এক্সিকিউট হবে। যাইহোক, যদি আমাদের নির্ধারিত কিউয়ের অনুরূপ রেডি কিউতে একাধিক প্রসেস নির্ধারণকারী দ্বারা নির্ধারিত হয়, তাহলে একটি দ্বিতীয় সিডিউলিং অ্যালগরিদম এই প্রসেসগুলি থেকে সেটটি বাছাই বা নির্বাচন করতে ব্যবহার করা হবে। উদাহরণস্বরূপ, আসুন আমরা বলতে পারি যে সিডিউলার অগ্রগতি ক্লাস নির্বাচন করার জন্য নির্ধারিত সময় নির্ধারণকারীর সিদ্ধান্ত নেওয়া হয়েছে, কারণ বাস্তব সময়ে এবং সিস্টেম অগ্রাধিকার ক্লাসগুলিতে কোনও প্রসেস তৈরি করা হয়নি। উপরন্তু, আসুন আমরা অনুমান করি যে ইন্টারেক্টিভ ক্লাসে 4 টি প্রসেস উপস্থিত রয়েছে। এখন একটি দ্বিতীয় রাউন্ড রবিন বা একটি FIFO সিডিউলিং অ্যালগরিদম এক্সিকিউট করে তারপর এই চারটি প্রসেসের কোনটি সিপিইউ বরাদ্দ করা হবে তা নির্ধারণ করতে চালানো হবে। সুতরাং, মাল্টিলেভেল

কিউতে আছে দুটি নির্ধারিত অ্যালগরিদম। প্রথমটি হল একটি অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম, যা অগ্রাধিকার শ্রেণির একটি বেছে নেয় এবং দ্বিতীয়টি একটি নন প্রায়রীটি ভিত্তিক অ্যালগরিদম, যা এই অগ্রাধিকার শ্রেণিতে প্রসেসগুলির মধ্যে বেছে নেয়। দ্বিতীয় সিডিউলিং অ্যালগরিদম সাধারণত রাউন্ড রবিন হবে, উচ্চতর অগ্রগতির দ্বিতীয় সিডিউলিং অ্যালগরিদম প্রসেস করে যা সাধারণত ফার্স্ট কাম ফার্স্ট সার্ভ হয়।

(স্লাইডটাইপ পড়ুন: 15:36)

আরও, নির্ধারিত সিডিউলার যে বহুমাত্রিক কিউ সমর্থন করে, সেটি সিডিউলার অগ্রাধিকার শ্রেণির উপর ভিত্তি করে সময় স্লাইসের সমন্বয় সাধন করতে পারে। সুতরাং, I / O বাউন্ড প্রসেসগুলি সাধারণত একটি উচ্চতর অগ্রাধিকার ক্লাস এবং একটি দীর্ঘ সময় স্লাইস হয়। এটি নিশ্চিত করে যে I / O বাউন্ড প্রসেসগুলি যত তাড়াতাড়ি সম্ভব CPU দ্বারা পরিবেশন করা হবে। এছাড়াও দীর্ঘ সময়সীমার স্লাইস সম্পূর্ণ হওয়ার আগেই I / O বাউন্ড প্রসেসের বার্ষ্টটি সম্পূর্ণ হয়। অন্যদিকে CPU বাউন্ড প্রসেসগুলিকে একটি নিম্ন অগ্রাধিকার ক্লাস বরাদ্দ করা হয় এবং সংক্ষিপ্ত সময় স্লাইস প্রদান করা হয়। সিডিউলিং অ্যালগরিদমের ওপর নির্ভর করে তৈরি একটি মাল্টিলেবেল সিডিউলিং কিউ থাকার প্রধান সমস্যা হল, প্রসেসটি শ্রেণির apriori বরাদ্দ করা আবশ্যিক যে আমরা একটি প্রসেস একটি CPU বাউন্ড প্রসেস বা একটি I / O বাউন্ড প্রসেস কিনা তা জানতে প্রয়োজন হবে। এখন এটা করা সহজ জিনিস নয়; প্রথমে এটি একটি প্রসেস প্রকৃতপক্ষে একটি I / O বাউন্ড প্রসেস বা একটি CPU বাউন্ড প্রসেস সনাক্ত করা খুব কঠিন। দ্বিতীয়ত, একক প্রসেসটি এক সময় I / O বাউন্ড প্রসেসের মতো কাজ করে, কিন্তু অন্য সময় এটি একটি CPU বাউন্ড প্রসেসের মত কাজ করতে পারে। যে, এক সময়ে এটি ছোট CPU বার্ষ্ট এবং নিষ্ক্রিয় সময়ের দীর্ঘ বার্ষ্ট থাকতে পারে, অন্য সময় এটি একটি CPU বাউন্ড প্রসেসের মত দীর্ঘ CPU বার্ষ্ট এবং সংক্ষিপ্ত I / O বিস্তারণ থাকতে পারে। অতএব, apriori সিদ্ধান্ত কিনা একটি নির্দিষ্ট প্রসেস I / O বাউন্ড প্রসেসকে একটি উচ্চ অগ্রাধিকার দেওয়া উচিত কঠিন।

(স্লাইডসময় পড়ুন: 17:24)

সুতরাং, এই নির্দিষ্ট সীমাবদ্ধতা কমানোর একটি উপায় মাল্টিলেবেল ফীডব্যাক কিউ ব্যবহার যা সিডিউলারে ব্যবহার করা হয়। এই সময়সূচীগুলির মধ্যে প্রসেসগুলি গতিশীলভাবে তার CPU, I / O কার্যকলাপের উপর নির্ভর করে অগ্রাধিকার শ্রেণিতে স্থানান্তরিত হয়। সুতরাং এই নির্দিষ্ট সময়সূচী এই মৌলিক পর্যবেক্ষণ কাজ করে যে CPU বাউন্ড প্রসেস তার সম্পূর্ণ সময় স্লাইস গণনা করে। সম্ভবত এই কারণে এটি একটি CPU বাউন্ড প্রসেসের CPU-র একটি দীর্ঘ বার্ষ্ট হবে এবং এটি সম্পূর্ণ সময় স্লাইস পর্যন্ত ব্যবহার করা হবে। অন্যদিকে I / O বাউন্ড প্রসেসগুলির মধ্যে একটি খুব সংক্ষিপ্ত CPU বার্ষ্ট সময় রয়েছে এবং তাই এটি সম্পূর্ণ সময় স্লাইড সম্পূর্ণ

করার সম্ভাবনা নেই। সুতরাং যদি আপনি এই বিশেষ উদাহরণ গ্রহণ করেন, আমরা বলতে পারি যে প্রসেস 1 এবং 4 তাদের সময় স্লাইস সম্পন্ন করেছে; অতএব এই প্রসেসগুলি CPU বাউন্ড হতে পারে। অন্য দিকে, প্রসেস 2 এখানে খুব ছোট CPU বাস্ট এখানে পাশাপাশি এখানে; অতএব, প্রসেস 2 একটি CPU বাউন্ড প্রসেসের পরিবর্তে একটি I / O বাউন্ড প্রসেস হতে পারে।

(স্লাইডটাইপ পড়ুন: 18:41)

একটি মাল্টিলেভেল ফিডব্যাক ক্যু ভিত্তিক সিডিউলিং অ্যালগরিদম এর মূল ধারণা হল নিম্নোক্ত; সব প্রসেস সর্বোচ্চ অগ্রাধিকার বা একটি বেস অগ্রাধিকার সঙ্গে শুরু করে। এখন যদি প্রসেসটি তার সময় স্লাইস শেষ করে, তবে যদি প্রসেসটি তার সময় স্লাইসের সমাপ্ত না হওয়া পর্যন্ত এক্সিকিউট হয় তাহলে এই ধারণাটি হবে যে প্রসেসটি একটি CPU বাউন্ড প্রসেস। তাই এটি ক্লাসে পরবর্তী নিম্ন অগ্রাধিকারে স্থানান্তরিত হয়। অন্যদিকে যদি প্রসেসটি তার সময়টি সম্পূর্ণ না করে তবে এটি অনুমান করা যায় যে প্রসেসটি একটি I / O বাউন্ড প্রসেস এবং তাই এটি অগ্রাধিকার শ্রেণিতে উচ্চতর অগ্রাধিকার শ্রেণিতে বৃদ্ধি করবে বা একই অগ্রাধিকার রাখবে। সুতরাং আপনি দেখতে পারেন এটি একটি প্রসেস অগ্রাধিকার ক্লাস পরিবর্তন করার একটি গতিশীল উপায়। তবে, সমস্যা হচ্ছে যে এখানেও স্টার্ভেশন হয় এবং তারপর কৌশলগুলি প্রয়োগ করা হয় যেখানে স্টার্ভেশনগুলি মোকাবেলা করার প্রয়োজন হয়।

(স্লাইডটাইম পড়ুন: 19:46)

এই নির্দিষ্ট সময়সূচী অ্যালগরিদমের আরেকটি সমস্যা হল, একটি দূষিত প্রসেস সিস্টেমে খেলা করে এবং সবসময় উচ্চ অগ্রাধিকার ক্লাস থাকতে পারে। সুতরাং, একটি উদাহরণ সঙ্গে এটি কিভাবে কাজ করে দেখুন। তাই আসুন আমরা দেখি যে দূষিত প্রসেসটি সিস্টেমের সময় স্লাইডটি জানে, অর্থাৎ এটি জানে যে কখন সময় স্লাইসটি সম্পূর্ণ হয়ে যায় এবং কখন একটি কন্ট্রোল সূইচ ঘটে। সুতরাং আসুন ওটার এটি একটি loop আছে যেমন এখানে রয়েছে। মূলত, দূষিত প্রসেস বেশিরভাগ সময় স্লাইসের জন্য কিছু কাজ করে এবং তারপর এটি সময় স্লাইসের শেষ পর্যন্ত ঘুমিয়ে থাকে। এখন যখন আমরা জানি একটি প্রসেস ঘুমাতে গেলে, এটি রানিং স্টেট থেকে ব্লক স্টেটে যায়। অতএব, স্লীপ একটি কন্ট্রোল সূইচ ঘটতে বাধ্য করবে। অতএব, কি ঘটতে যাচ্ছে তা হল যে একটি নতুন প্রসেস বাকি সময় স্লাইসের জন্য চালানো হবে। এইভাবে, পুরো সময় স্লাইসটি এখানে উদাহরণস্বরূপ প্রসেস 4 এর জন্য যা একটি দারুণ প্রসেস যা অধিকাংশ সময় স্লাইসের জন্য সঞ্চালিত হবে এবং তারপর সময় স্লাইস সমাপ্ত হওয়ার আগে এটি ঘুমাতে যায়; এইভাবে, একটি অন্য প্রসেসকে এক্সিকিউট করতে বাধ্য করে। অন্য প্রসেস 1 এখন একটি ছোট সময়কালের জন্য এক্সিকিউট করবে। সময় স্লাইস শেষ হওয়ার সময় সিডিউলারটি দেখে যে প্রসেস 1 টির সময় স্লাইস সম্পন্ন হয়েছে, তাই এটি অনুমান করা যাচ্ছে যে প্রসেস 1 CPU বাউন্ড প্রসেস এবং প্রসেস 1টি কম অগ্রাধিকার শ্রেণিতে সরানো হয়েছে। অন্যদিকে, প্রসেস 4 যা ঘুমের উপর ব্লক ছিল উচ্চ

অগ্রাধিকার শ্রেণীতে থাকবে। সুতরাং, প্রসেস 4 সিস্টেম খেলা করতে সক্ষম হবে। এটি উচ্চ অগ্রাধিকার ক্লাস থেকে এক্সিকিউসন চালিয়ে যাবে এবং অন্য প্রসেস যেমন প্রসেস 1 নিম্ন অগ্রাধিকার ক্লাসে স্থানান্তরিত হবে। ধন্যবাদ।